

ITRI625 project

Lecturer: Prof. Lynette Drevin and Mr. Michael de Jager

Facilitator: Mr. Bernard Swanepoel

Date: TBD

1 Introduction

This project is presented to the student to provide experience in the fields of Machine Learning (ML) and Deep Learning (DL) within the context of security. For this project, you will choose one of the datasets listed below, or propose your own dataset within the context of security, and develop a model that aims to solve a certain problem in the context of cybersecurity. After this, the student will implement the model into a basic desktop app, allowing the model to be practically usable. The project consists of 70% practical work and 30% theoretical analysis.

List of datasets:

- [Phishing email dataset](#): Can be used to train a model to detect phishing messages.
- [Intrusion detection dataset](#): Used to build a model that can do intrusion detection.
- [Cyber Threat Dataset: Network, Text and Relation](#): Can be used to classify cybersecurity threats.
- Propose your own cybersecurity dataset by emailing it to [Bernard](#) for approval.

NOTE!

- Two students may work in a group.
- All model code should be written in Python within a Jupyter Notebook (.ipynb), and all figures and evaluation metrics should be viewable directly within the notebook.
- GitHub or any other source control should be used throughout this project.

2 Practical part (70 marks)

2.1 Development of model (20 marks)

After choosing one of the datasets, students should develop a model that uses the dataset to solve a certain problem. Models can be developed using TensorFlow, PyTorch or any other relevant ML or DL library/framework.

2.2 Dataset divided into sets (5 marks)

The dataset should be divided into a train, validation and testing set.

2.3 Logging of evaluation metrics (15 marks)

All relevant evaluation metrics should be logged after each iteration. Example, for the task of classification, the following metrics can be logged:

- Accuracy
- Precision
- Recall
- F1-score
- AUC-ROC (Area Under the Receiver Operating Characteristic curve)
- Loss

2.4 Metric graphs (15 marks)

All relevant metric graphs should be plotted using Matplotlib. Example, for the task of classification, the following metric graphs can be sketched:

- ROC Curve (Receiver Operating Characteristic curve)
- Precision–Recall (PR) Curve
- Confusion matrix
- Loss curve

2.5 Desktop application (15 marks)

Create a basic API for the model using Flask or FastAPI and implement the model's endpoint into a basic desktop app using Tkinter. Other technologies can be used but this is by far the simplest approach.

2.6 Patience/early stopping (5 marks)

Implement a patience value in the main training loop to prevent the model from overtraining and overfitting. The patience value ensures that early stopping is triggered when the model's performance metrics no longer improve, stopping training before overfitting occurs.

2.7 Additional features (10 marks)

Bonus marks will be awarded for implementing additional techniques like XAI (Explainable Artificial Intelligence), hosting the model or any other extra efforts. Marks cannot exceed 100% of the total project score.

3 Theory component (30 marks)

3.1 Model scenario (3 marks)

Give the scenario of your model. How is this model useful in the context of cybersecurity?

3.2 Model architecture (10 marks)

Explain all relevant details of the model you used. For example, if you implemented a Convolutional Neural Network (CNN), specify the types of layers, the activation functions used, and the overall model architecture. Also explain why the chosen model is well suited to the data it was trained on.

3.3 Model result analysis (10 marks)

Explain the results of your model using the metrics and graphs derived from the model's training. Is this model effective in the problem it is trying to resolve?

3.4 Real-world use of models (7 marks)

Evaluate the real-world use of models within the field of cybersecurity, focusing on how these models are applied to tasks such as intrusion detection, threat classification, anomaly detection, and risk assessment. Discuss their effectiveness in practical deployments, including strengths, limitations, and common challenges such as data quality, concept drift, and false positives. Use academic references.

4 Summary of marks

Component	Description	Marks
Total Practical Marks: 70		
Model development	Develop a machine learning or deep learning model to solve a cybersecurity-related problem using a selected dataset	20
Dataset preparation	Correctly divide the dataset into training, validation, and testing sets	5
Evaluation metric logging	Log relevant evaluation metrics during training (e.g. accuracy, precision, recall, F1-score, AUC-ROC, loss)	15
Metric visualization	Plot relevant metric graphs such as ROC curves, precision-recall curves, confusion matrices, and loss curves	15
Desktop application	Expose the model via an API and integrate it into a basic desktop application for practical use	15
Early stopping	Implement patience-based early stopping to prevent overfitting during training	5
Additional features	Bonus marks for advanced techniques like XAI or hosting a model	10
Total Theory Marks: 30		
Model scenario	Describe the model scenario and explain its relevance within the cybersecurity domain	3
Model architecture	Explain the model architecture, including layers, activation functions, and justification for the chosen design	10
Model result analysis	Analyse the model's performance using evaluation metrics and graphs, and discuss its effectiveness	10
Real-world use of models	Evaluate the real-world use of machine learning models in cybersecurity, supported by academic or industry references	7
Deductions (if applicable)		
Missing notebook outputs	Figures, metrics, or results not visible within the Jupyter Notebook	-10
Poor version control	Lack of consistent use of GitHub or equivalent source control	-10
Total for this project: 100		